Challan Management System — Project Context

Project Overview

A **dynamic challan management system** built on Cloudflare's edge infrastructure. It enables businesses to create, track, and transfer supply-chain challans (dispatch slips) across a multi-vendor chain with granular Role-Based Access Control (RBAC), damage tracking, and comprehensive reporting.

Tech Stack

Layer	Technology	Purpose
Runtime	Cloudflare Workers	Serverless edge API (globally distributed)
Database	Cloudflare D1 (SQLite)	Relational data — challans, vendors, users, RBAC
File Storage	Cloudflare R2	Attachment blobs, generated PDF reports
Auth	Cloudflare Workers + JWT	Stateless session tokens
Frontend	React + Vite (deployed via Cloudflare Pages)	SPA UI
Styling	Tailwind CSS	Utility-first styling
PDF Generation	`@react-pdf/renderer` or `pdfmake` (client-side)	Challan/report PDFs
Validation	Zod	Schema validation on API boundary
ORM/Query	Drizzle ORM (D1 adapter)	Type-safe SQL queries

Core Domain Concepts

Challan

A dispatch record linking a source vendor to products being transferred. Contains:

- Challan number (auto-generated, unique)
- Source vendor
- Date of creation
- Line items (product name, quantity, price per unit)
- Current status: `DRAFT → ACTIVE → IN_TRANSIT → DELIVERED → CLOSED`
- Transfer history log

Vendor

Any entity in the supply chain (manufacturer, distributor, retailer). Has:

- Unique vendor code
- Contact details
- Assigned users
- Challan history (sent + received)

Transfer

A challan moves from one vendor to the next. Each transfer:

- Records the sender and receiver vendor
- Timestamps the handoff
- Allows the receiver to adjust quantities/prices before accepting
- Creates an immutable audit entry

Damage Record

At each transfer stage, the receiving vendor may log damaged units:

- Linked to a specific challan + transfer leg
- Product-level granularity (damaged quantity per line item)
- Automatically deducted from received quantity

RBAC (Role-Based Access Control)

Roles are defined per-organization and assigned per-user:

Role	Capabilities
`SUPER_ADMIN`	Full system access, user management, org settings

Role	Capabilities
`ADMIN`	Vendor management, all challans, reports
`MANAGER`	Create/transfer challans within assigned vendors
`OPERATOR`	View and accept challans for their vendor
`VIEWER`	Read-only access to assigned challans

Permissions are scoped: a user may be a `MANAGER` for Vendor A but `VIEWER` for Vendor B.

Database Schema (D1 / SQLite)

```
-- Organizations (multi-tenant)
organizations (id, name, slug, created_at)

-- Users
users (id, org_id, email, password_hash, name, created_at)

-- Vendors
vendors (id, org_id, vendor_code, name, address, phone, created_at)

-- User-Vendor role assignments (RBAC)
user_vendor_roles (id, user_id, vendor_id, role, created_at)

-- Challans
challans (id, org_id, challan_number, source_vendor_id, status,
          notes, created_by, created_at, updated_at)

-- Challan line items
challan_items (id, challan_id, product_name, quantity, unit_price,
               current_quantity, current_unit_price)

-- Transfer legs
transfers (id, challan_id, from_vendor_id, to_vendor_id,
           transferred_by, accepted_by, status, notes,
           transferred_at, accepted_at)

-- Transfer item adjustments (receiver can edit qty/price)
transfer_item_adjustments (id, transfer_id, challan_item_id,
                           adjusted_quantity, adjusted_unit_price,
                           adjustment_reason)
```

```
-- Damage records
damage_records (id, transfer_id, challan_item_id, damaged_quantity,
                notes, reported_by, created_at)

-- Audit log
audit_log (id, org_id, user_id, action, entity_type, entity_id,
           payload_json, ip_address, created_at)
```

API Design (Cloudflare Workers)

Base URL: `https://api.challan.yourdomain.com`

Auth

```
POST   /auth/login
POST   /auth/logout
POST   /auth/refresh
GET    /auth/me
```

Challans

```
GET    /challans                  List (filters: status, vendor, date range)
POST   /challans                  Create new challan
GET    /challans/:id              Get challan detail + items + history
PUT    /challans/:id              Update draft challan
DELETE /challans/:id              Delete draft challan
POST   /challans/:id/transfer     Initiate transfer to next vendor
POST   /challans/:id/accept       Receiving vendor accepts (with adjustments)
POST   /challans/:id/damage       Log damage for a transfer leg
GET    /challans/:id/pdf          Generate & return PDF (stored in R2)
```

Vendors

```
GET    /vendors
POST   /vendors
GET    /vendors/:id
PUT    /vendors/:id
```

```
DELETE /vendors/:id
GET    /vendors/:id/challans
```

Users & RBAC

```
GET    /users
POST   /users
PUT    /users/:id/roles          Assign/update role for a vendor
DELETE /users/:id
```

Reports

```
GET    /reports/challans         Challan summary report
GET    /reports/damages          Damage report (by vendor, date, product)
GET    /reports/vendor-performance Vendor performance metrics
GET    /reports/audit-log        Full audit trail
POST   /reports/export           Export report as PDF (stored in R2,
returns URL)
```

File Storage (R2)

Bucket structure:

```
challan-attachments/
  {org_id}/
    challans/
      {challan_id}/
        {file_name}          ← attachments uploaded to a challan

    reports/
      {report_id}.pdf        ← generated report PDFs

    exports/
      {export_id}.pdf        ← challan PDF exports
```

Key Business Rules

1. A challan can only be transferred if its status is `ACTIVE`.
2. Only the **current holding vendor** can initiate a transfer.

3. The **receiving vendor** must accept or reject within the system before status advances.
4. Price/quantity adjustments on acceptance are logged but do **not** overwrite originals — both original and adjusted values are retained.
5. Damage records are append-only — once logged they cannot be deleted, only annotated.
6. A challan reaches `CLOSED` status once the final vendor marks it delivered.
7. All mutations are recorded in `audit_log` with full payload snapshots.
8. Reports are always generated from the database — not from PDFs — ensuring accuracy.
9. RBAC checks happen at the Worker middleware level, before any route handler executes.

Project Folder Structure

```
/
├── apps/
│   ├── web/                          ← React + Vite frontend (Cloudflare
Pages)
│   │   ├── src/
│   │   │   ├── pages/
│   │   │   ├── components/
│   │   │   ├── hooks/
│   │   │   ├── stores/               ← Zustand state
│   │   │   ├── api/                  ← Typed API client (fetch wrappers)
│   │   │   └── utils/
│   │   └── public/
│   │
│   └── api/                          ← Cloudflare Workers API
│       ├── src/
│       │   ├── routes/
│       │   │   ├── auth.ts
│       │   │   ├── challans.ts
│       │   │   ├── vendors.ts
│       │   │   ├── users.ts
│       │   │   └── reports.ts
│       │   ├── middleware/
│       │   │   ├── auth.ts           ← JWT verification
│       │   │   ├── rbac.ts           ← Permission checks
│       │   │   └── logger.ts
│       │   ├── db/
│       │   │   ├── schema.ts         ← Drizzle schema definitions
│       │   │   ├── migrations/
│       │   │   └── queries/          ← Reusable typed queries
```

```
│       │   ├── services/
│       │   │   ├── challan.service.ts
│       │   │   ├── transfer.service.ts
│       │   │   ├── damage.service.ts
│       │   │   ├── report.service.ts
│       │   │   └── r2.service.ts
│       │   ├── utils/
│       │   └── index.ts              ← Worker entry point (Hono router)
│       └── wrangler.toml
│
├── packages/
│   └── shared/                       ← Shared Zod schemas + TypeScript
types
│       ├── schemas/
│       └── types/
│
└── README.md
```

Development Notes

- Use **Hono** as the HTTP router inside the Worker (lightweight, type-safe, edge-native).
- Use **Drizzle ORM** with the `d1` adapter for all database access — never raw SQL strings.
- All API responses follow the shape: `{ success: boolean, data?: T, error?: string, meta?: PaginationMeta }`.
- Use **Zod** on every POST/PUT body — reject with 400 before business logic.
- JWT secret is stored in Cloudflare Workers **Secrets** (not `wrangler.toml`).
- R2 presigned URLs expire in 1 hour for report downloads.
- D1 migrations are managed with `drizzle-kit` and run via `wrangler d1 migrations apply`.
- Frontend is deployed on **Cloudflare Pages** with environment variables injected at build time.
- All timestamps stored as ISO 8601 UTC strings in D1.

Deployment Checklist

- [ ] Create D1 database: `wrangler d1 create challan-db`
- [ ] Create R2 bucket: `wrangler r2 bucket create challan-attachments`
- [ ] Set Worker secrets: `JWT_SECRET`, `ADMIN_SEED_EMAIL`, `ADMIN_SEED_PASSWORD`
- [ ] Run migrations: `wrangler d1 migrations apply challan-db`

- [ ] Deploy Worker API: `wrangler deploy`
- [ ] Deploy frontend: `wrangler pages deploy apps/web/dist`
- [ ] Set CORS origins in Worker to match Pages domain
- [ ] Configure custom domain in Cloudflare dashboard